

CPSC 375 - FINAL PROJECT

GROUP MEMBERS: CELESTE GARLEJO, KATELYN NGUYEN

Introduction

The goal of this project is to predict COVID-19 deaths two weeks ahead using linear regression models and demographic information from multiple datasets. COVID-19 data from Our World in Data and population estimates from the World Bank were combined to analyze how healthcare, demographics, and case counts affect future death rates.

✓ Data Wrangling

The COVID-19 and World Bank datasets were cleaned and merged into a single DataFrame for analysis.

Only country-level observations were kept by removing rows where the ISO country code was not exactly three characters long. Countries with populations below one million were also removed to reduce noise from very small populations.

The `date` column was converted to datetime format, and a new variable called `new_deaths_smoothed_2wk` was created by shifting COVID-19 deaths forward by 14 days within each country.

The World Bank dataset was transformed from wide format into tidy format so demographic variables could be merged with the COVID-19 dataset. The datasets were then merged together using country and year information to create the final modeling dataset.

A. Read the two data files using `pd.read_csv()`

```

1 import pandas as pd
2
3 codebook = pd.read_csv(
4     "https://raw.githubusercontent.com/owid/covid-19-data/master/p
5 )
6 codebook.head()

```

	column	source	category	description	
0	iso_code	International Organization for Standardization	Others	ISO 3166-1 alpha-3 – three-letter country code...	
1	continent	Our World in Data	Others	Continent of the geographical location	
2	location	Our World in Data	Others	Geographical location	
3	date	Our World in Data	Others	Date of observation	

Next steps:

[Generate code with codebook](#)

[New interactive sheet](#)

```

1 data = pd.read_csv(
2     "https://raw.githubusercontent.com/owid/covid-19-data/master/p
3 )
4 data.head()

```

	iso_code	continent	location	date	total_cases	new_cases	new_cas
0	AFG	Asia	Afghanistan	2020-01-05	0.0	0.0	
1	AFG	Asia	Afghanistan	2020-01-06	0.0	0.0	
2	AFG	Asia	Afghanistan	2020-01-07	0.0	0.0	
3	AFG	Asia	Afghanistan	2020-01-08	0.0	0.0	
4	AFG	Asia	Afghanistan	2020-01-09	0.0	0.0	

5 rows x 67 columns

Description

The OWID COVID dataset and codebook were loaded directly from GitHub using `pd.read_csv()`.

B. Keep only country-level data by removing all rows where `iso_code` is not exactly 3 letters.

(The remaining items represent larger regions like continents.)

Hint: `.str.len()` returns the number of characters.

```
1 data = data[data["iso_code"].str.len() == 3]
2 data
```

	iso_code	continent	location	date	total_cases	new_cases	new_deaths
0	AFG	Asia	Afghanistan	2020-01-05	0.0	0.0	0.0
1	AFG	Asia	Afghanistan	2020-01-06	0.0	0.0	0.0
2	AFG	Asia	Afghanistan	2020-01-07	0.0	0.0	0.0
3	AFG	Asia	Afghanistan	2020-01-08	0.0	0.0	0.0
4	AFG	Asia	Afghanistan	2020-01-09	0.0	0.0	0.0
...	...	...	...	...	...	...	...
429430	ZWE	Africa	Zimbabwe	2024-07-31	266386.0	0.0	0.0
429431	ZWE	Africa	Zimbabwe	2024-08-01	266386.0	0.0	0.0
429432	ZWE	Africa	Zimbabwe	2024-08-02	266386.0	0.0	0.0
429433	ZWE	Africa	Zimbabwe	2024-08-03	266386.0	0.0	0.0
429434	ZWE	Africa	Zimbabwe	2024-08-04	266386.0	0.0	0.0

395311 rows × 67 columns

Description

Rows representing continents or larger regions were removed by keeping only ISO codes with exactly 3 letters.

C. Remove countries whose total population is less than 1 million

```
1 data = data[data["population"] >= 1_000_000]
2 data
```

	iso_code	continent	location	date	total_cases	new_cases	new_deaths
0	AFG	Asia	Afghanistan	2020-01-05	0.0	0.0	0.0
1	AFG	Asia	Afghanistan	2020-01-06	0.0	0.0	0.0
2	AFG	Asia	Afghanistan	2020-01-07	0.0	0.0	0.0
3	AFG	Asia	Afghanistan	2020-01-08	0.0	0.0	0.0
4	AFG	Asia	Afghanistan	2020-01-09	0.0	0.0	0.0
...	...	...	...	...	...	...	...
429430	ZWE	Africa	Zimbabwe	2024-07-31	266386.0	0.0	0.0
429431	ZWE	Africa	Zimbabwe	2024-08-01	266386.0	0.0	0.0
429432	ZWE	Africa	Zimbabwe	2024-08-02	266386.0	0.0	0.0
429433	ZWE	Africa	Zimbabwe	2024-08-03	266386.0	0.0	0.0
429434	ZWE	Africa	Zimbabwe	2024-08-04	266386.0	0.0	0.0

266897 rows × 67 columns

Description

Removes row of countries with populations under 1 million were also removed.

D. Add a new column `new_deaths_smoothed_2wk` that has the same values as `new_deaths_smoothed` but two weeks ahead. This will be used for linear modeling as described below.

Hint: Pandas has a `pd.to_datetime()` function and `pd.Timedelta` object that together enable calculations with dates like `.assign(date=date - pd.Timedelta(14))` and `.query('date >= "2023-01-01")`.

```
1 data["date"] = pd.to_datetime(data["date"])
2
3 future = (
4     data[["location", "date", "new_deaths_smoothed"]]
5     .assign(date=lambda x: x["date"] - pd.Timedelta(days=14))
6     .rename(columns={"new_deaths_smoothed": "new_deaths_smoothed_2w
7 )
8
9 data = data.merge(future, on=["location", "date"], how="left")
10 data
```

/tmp/ipykernel_5904/3419478451.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using `.loc[row_indexer,col_indexer] = value` instead

See the caveats in the documentation: [https://pandas.pydata.org/pandas-](https://pandas.pydata.org/pandas-data/)
`data["date"] = pd.to_datetime(data["date"])`

	iso_code	continent	location	date	total_cases	new_cases	new_deaths
0	AFG	Asia	Afghanistan	2020-01-05	0.0	0.0	0.0
1	AFG	Asia	Afghanistan	2020-01-06	0.0	0.0	0.0
2	AFG	Asia	Afghanistan	2020-01-07	0.0	0.0	0.0
3	AFG	Asia	Afghanistan	2020-01-08	0.0	0.0	0.0
4	AFG	Asia	Afghanistan	2020-01-09	0.0	0.0	0.0
...	...	...	...	...	...	...	...
268906	ZWE	Africa	Zimbabwe	2024-07-31	266386.0	0.0	0.0
268907	ZWE	Africa	Zimbabwe	2024-08-01	266386.0	0.0	0.0

268908	ZWE	Africa	Zimbabwe	2024-08-02	266386.0	0.0
268909	ZWE	Africa	Zimbabwe	2024-08-03	266386.0	0.0
268910	ZWE	Africa	Zimbabwe	2024-08-04	266386.0	0.0

268911 rows x 68 columns

Description

A new target variable called `new_deaths_smoothed_2wk days` was created by shifting deaths forward by 14. This allows the model to predict future deaths.

E. Tidy tables, as needed.

Hint: only the population data is not tidy.

```

1 data_tidy = data.copy()
2
3 data_tidy["date"] = pd.to_datetime(data_tidy["date"])
4 data_tidy["year"] = data_tidy["date"].dt.year
5
6 data_tidy = data_tidy.groupby(
7     ["iso_code", "continent", "location", "year"],
8     as_index=False
9 ).agg({
10     "population": "first",
11     "total_cases": "mean",
12     "new_cases": "mean",
13     "new_deaths": "mean",
14     "total_deaths": "mean"
15 })
16
17 data_tidy

```

	iso_code	continent	location	year	population	total_cases	new_
0	AFG	Asia	Afghanistan	2020	41128772	22886.718232	143.2
1	AFG	Asia	Afghanistan	2021	41128772	107335.397260	290.5
2	AFG	Asia	Afghanistan	2022	41128772	185140.610959	135.3
3	AFG	Asia	Afghanistan	2023	41128772	219893.526027	63.7
4	AFG	Asia	Afghanistan	2024	41128772	233590.364055	22.2
...	...	...	...	...	...	...	...
789	ZWE	Africa	Zimbabwe	2020	16320539	3546.859116	35.8
790	ZWE	Africa	Zimbabwe	2021	16320539	80403.690411	522.6
791	ZWE	Africa	Zimbabwe	2022	16320539	249333.298630	153.9
792	ZWE	Africa	Zimbabwe	2023	16320539	264841.627397	16.8
793	ZWE	Africa	Zimbabwe	2024	16320539	266337.161290	1.4

794 rows x 9 columns

Next steps:

[Generate code with data_tidy](#)

[New interactive sheet](#)

F. Merge the two data tables.


```

1 data["year"] = data["date"].dt.year
2 data_merged = data.merge(
3     data_tidy,
4     on=["location", "year"],
5     how="left"
6 )
7
8 data_merged

```

	iso_code_x	continent_x	location	date	total_cases_x	new_cas
0	AFG	Asia	Afghanistan	2020-01-05	0.0	
1	AFG	Asia	Afghanistan	2020-01-06	0.0	
2	AFG	Asia	Afghanistan	2020-01-07	0.0	
3	AFG	Asia	Afghanistan	2020-01-08	0.0	
4	AFG	Asia	Afghanistan	2020-01-09	0.0	
...	...	...	...	...	...	...
268906	ZWE	Africa	Zimbabwe	2024-07-31	266386.0	
268907	ZWE	Africa	Zimbabwe	2024-08-01	266386.0	
268908	ZWE	Africa	Zimbabwe	2024-08-02	266386.0	
268909	ZWE	Africa	Zimbabwe	2024-08-03	266386.0	
268910	ZWE	Africa	Zimbabwe	2024-08-04	266386.0	

268911 rows x 76 columns

✓ Choosing Predictor Variables

Predictor variables were selected based on prior knowledge about factors that may affect COVID-19 death rates.

The primary predictor used was `new_cases_smoothed` because increases in COVID-19 cases are expected to lead to increased deaths approximately two weeks later.

Healthcare-related variables such as `hosp_patients`, `hospital_beds_per_thousand`, and `diabetes_prevalence` were included because countries with weaker healthcare systems or higher rates of pre-existing conditions may experience higher mortality.

Demographic and vaccination-related variables such as `older_population` and `vaccinations_per_person` were also tested because elderly populations are more vulnerable to severe COVID-19 outcomes and vaccination rates may reduce death rates.

Different combinations of variables were tested across multiple models to compare predictive performance using R^2 and RMSE values.

✓ Variable Transformations

Several transformed variables were created to improve the regression models and better represent relationships between demographics, healthcare, and COVID-19 deaths.

cardiovasc_deaths

This variable was created by multiplying cardiovascular death rate by total population to estimate the overall impact of cardiovascular disease within each country.

older_population

This variable estimates the elderly population by multiplying the percentage of people age 65 and older by total population. Older populations are more vulnerable to severe COVID-19 outcomes.

hospital_beds_total

This variable estimates total hospital bed capacity by multiplying hospital beds per thousand people by total population.

vaccinations_per_person

This variable was created by dividing total vaccinations by total population in order to compare vaccination coverage across countries of different sizes.

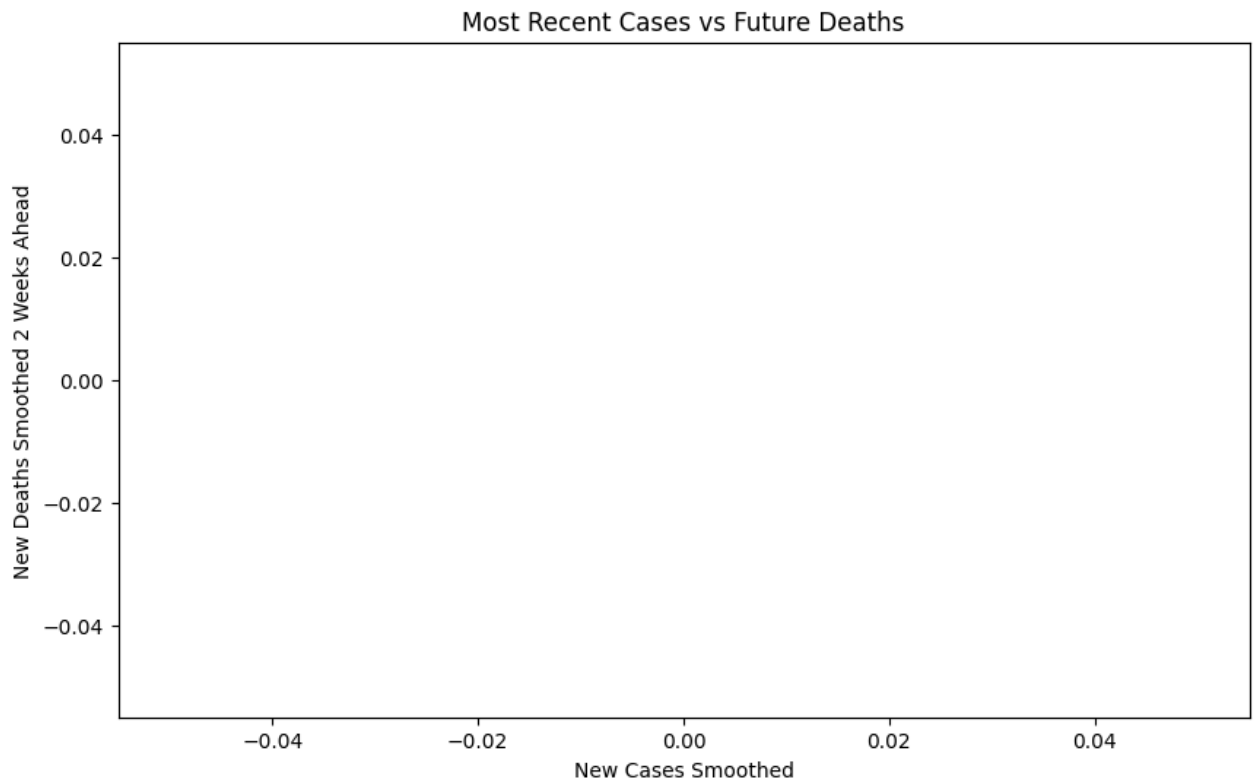
cases_per_person

This variable was created by dividing total COVID cases by total population to estimate infection spread relative to population size.

✓ Scatterplots

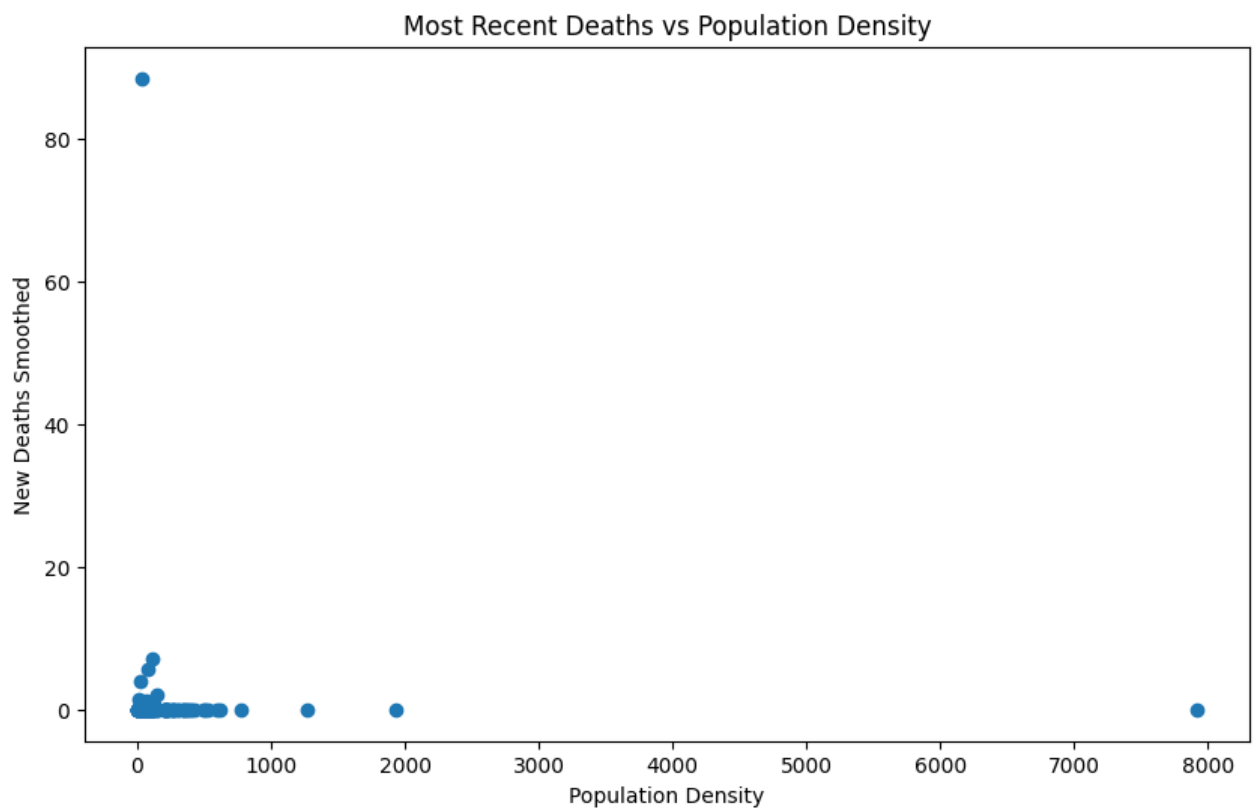
- Scatterplot of only the most recently available `new_deaths_smoothed_2wk` and `new_cases_smoothed` for every country

```
1 import matplotlib.pyplot as plt
2 latest_data = (
3     data.sort_values("date")
4     .groupby("location")
5     .tail(1)
6     )
7
8 plt.figure(figsize=(10, 6))
9 plt.scatter(
10 latest_data["new_cases_smoothed"],
11 latest_data["new_deaths_smoothed_2wk"]
12 )
13 plt.xlabel("New Cases Smoothed")
14 plt.ylabel("New Deaths Smoothed 2 Weeks Ahead")
15 plt.title("Most Recent Cases vs Future Deaths")
16 plt.show()
```



This scatterplot compares current COVID case counts with future deaths for each country. This scatterplot is currently blank because the data for of only the most recently available `new_deaths_smoothed_2wk` for every country is either 0 or NaN

```
1 plt.figure(figsize=(10, 6))
2 plt.scatter(
3 latest_data["population_density"],
4 latest_data["new_deaths_smoothed"]
5 )
6 plt.xlabel("Population Density")
7 plt.ylabel("New Deaths Smoothed")
8 plt.title("Most Recent Deaths vs Population Density")
9 plt.show()
```



This scatterplot compares recent deaths with population density for every country. only the most recent new deaths per day and the urban population

✓ Linear Modeling

- to develop a model to predict `new_deaths_smoothed` two weeks in the future, using the other variables, including any transformations.

A. Make a list of all predictor variables that are available. The challenge is to identify which combination of these predictors will give the best predictive model.

```
1 predictor_variables = [  
2     "new_cases_smoothed",  
3     "gdp_per_capita",  
4     "hosp_patients",  
5     "diabetes_prevalence",  
6     "total_vaccinations",  
7     "population",  
8     "aged_65_older",  
9     "cardiovasc_death_rate",  
10    "hospital_beds_per_thousand"  
11 ]  
12  
13 predictor_variables
```

```
['new_cases_smoothed',  
'gdp_per_capita',  
'hosp_patients',  
'diabetes_prevalence',  
'total_vaccinations',  
'population',  
'aged_65_older',  
'cardiovasc_death_rate',  
'hospital_beds_per_thousand']
```

B. Generate some (at least 3) transformed variables. For example, these could combine variables

(e.g., `cardiovasc_deaths = cardiovasc_death_rate * population`).

```

1 # Transformation 1
2 data["cardiovasc_deaths"] = data["cardiovasc_death_rate"] * data["population"]
3
4 # Transformation 2
5 data["older_population"] = data["aged_65_older"] / 100 * data["population"]
6
7 # Transformation 3
8 data["hospital_beds_total"] = data["hospital_beds_per_thousand"] / 1000 * data["population"]
9
10 # Transformation 4
11 data["vaccinations_per_person"] = data["total_vaccinations"] / data["population"]
12
13 # Transformation 5
14 data["cases_per_person"] = data["total_cases"] / data["population"]
15
16 data[["cardiovasc_deaths", "older_population", "hospital_beds_total", "vaccinations_per_person", "cases_per_person"]]

```

	cardiovasc_deaths	older_population	hospital_beds_total	vaccinations_per_person
0	2.455511e+10	1.061122e+06	20564.386	
1	2.455511e+10	1.061122e+06	20564.386	
2	2.455511e+10	1.061122e+06	20564.386	
3	2.455511e+10	1.061122e+06	20564.386	
4	2.455511e+10	1.061122e+06	20564.386	

C. Split your dataset into train and test subsets: only data from 2022 should be used for building/training the linear models in `sm.formula.ols()`.

(Data from 2023 will be used for evaluation as described later).

Note: each day is one data point.

```

1 data["date"] = pd.to_datetime(data["date"])
2
3 train = data[data["date"].dt.year == 2022]
4 test  = data[data["date"].dt.year == 2023]

```

```
1 train.head()
```

	iso_code	continent	location	date	total_cases	new_cases	new_c
727	AFG	Asia	Afghanistan	2022-01-01	157902.0	0.0	
728	AFG	Asia	Afghanistan	2022-01-02	158112.0	210.0	
729	AFG	Asia	Afghanistan	2022-01-03	158112.0	0.0	
730	AFG	Asia	Afghanistan	2022-01-04	158112.0	0.0	
731	AFG	Asia	Afghanistan	2022-01-05	158112.0	0.0	

5 rows x 74 columns

```
1 test.head()
```

	iso_code	continent	location	date	total_cases	new_cases	new_c
1092	AFG	Asia	Afghanistan	2023-01-01	207579.0	257.0	
1093	AFG	Asia	Afghanistan	2023-01-02	207579.0	0.0	
1094	AFG	Asia	Afghanistan	2023-01-03	207579.0	0.0	
1095	AFG	Asia	Afghanistan	2023-01-04	207579.0	0.0	
1096	AFG	Asia	Afghanistan	2023-01-05	207579.0	0.0	

5 rows x 74 columns

D. Run linear regression with at least 5 different combinations of predictor variables.

Hint: each model will look something like:

```
new_deaths_smoothed_2wk ~ new_cases_smoothed + gdp_per_capita +  
diabetes_prevalence + icu_patients + Q("SP.URB.TOTL")
```

```
1 import statsmodels.api as sm  
2 # model 1  
3 model = sm.formula.ols(  
4     'new_deaths_smoothed_2wk ~ new_cases_smoothed + gdp_per_capita  
5     data=train  
6 )  
7 results = model.fit()  
8 print(results.summary())
```

OLS Regression Results

```
=====
```

Dep. Variable:	new_deaths_smoothed_2wk	R-squared:	
Model:	OLS	Adj. R-squared:	
Method:	Least Squares	F-statistic:	
Date:	Sun, 17 May 2026	Prob (F-statistic):	
Time:	03:37:34	Log-Likelihood:	
No. Observations:	6167	AIC:	
Df Residuals:	6161	BIC:	
Df Model:	5		
Covariance Type:	nonrobust		

```
=====
```

	coef	std err	t	P> t	[0.001, 0.01, 0.05, 0.1, 0.5, 1]
Intercept	-67.8077	4.835	-14.024	0.000	-7.000
new_cases_smoothed	0.0003	4.12e-05	6.807	0.000	0.000
gdp_per_capita	0.0003	0.000	2.500	0.012	5.400
hosp_patients	0.0151	0.000	65.064	0.000	0.000
diabetes_prevalence	5.2924	0.400	13.218	0.000	0.000
total_vaccinations	-5.385e-09	1.29e-08	-0.417	0.677	-3.000

```
=====
```

Omnibus:	4409.167	Durbin-Watson:	
Prob(Omnibus):	0.000	Jarque-Bera (JB):	24.000
Skew:	2.835	Prob(JB):	
Kurtosis:	33.393	Cond. No.	

```
=====
```

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is
- [2] The condition number is large, 7.07e+08. This might indicate that strong multicollinearity or other numerical problems.

✓ Evaluating the Linear Models

- by comparing their predicted number of daily deaths in each day in 2023 (the test data) and comparing it with the actual number of deaths on those days. Specifically:

A. For each of your models, calculate the Root Mean Squared Error (RMSE) over all days in 2023 and all countries.

Hint: use `sm.tools.eval_measures.rmse()` or `sklearn.metrics.root_mean_squared_error()`.

```
1 from sklearn.metrics import root_mean_squared_error, r2_score
2
3 test = test.copy()
4
5 test["pred"] = results.predict(test)
6
7 eval_data = test[["new_deaths_smoothed_2wk", "pred"]].dropna()
8
9 rmse = root_mean_squared_error(
10     eval_data["new_deaths_smoothed_2wk"],
11     eval_data["pred"]
12 )
13
14 rmse
```

54.339723810132305

```
1 # Create dictionary of model formulas
2 model_formulas = {
3
4     "Model 1":
5     "new_deaths_smoothed_2wk ~ new_cases_smoothed + gdp_per_capita
6
7     "Model 2":
8     "new_deaths_smoothed_2wk ~ new_cases_smoothed + hosp_patients +
9
10    "Model 3":
11    "new_deaths_smoothed_2wk ~ new_cases_smoothed + aged_65_older +
12
13    "Model 4":
14    "new_deaths_smoothed_2wk ~ new_cases_smoothed + vaccinations_pe
```

```

15
16     "Model 5":
17     "new_deaths_smoothed_2wk ~ new_cases_smoothed + cardiovasc_deat
18 }
19
20 # Store evaluation results
21 model_results = []
22
23 # Store fitted models
24 fitted_models = {}
25
26 # Train and evaluate each model
27 for name, formula in model_formulas.items():
28
29     # Fit model
30     model = sm.formula.ols(
31         formula=formula,
32         data=train
33     ).fit()
34
35     # Save model
36     fitted_models[name] = model
37
38     # Predict on test data
39     test[f"{name}_pred"] = model.predict(test)
40
41     # Remove missing values
42     eval_data = test[
43         [
44             "new_deaths_smoothed_2wk",
45             f"{name}_pred"
46         ]
47     ].dropna()
48
49     # Calculate RMSE
50     rmse = root_mean_squared_error(
51         eval_data["new_deaths_smoothed_2wk"],
52         eval_data[f"{name}_pred"]
53     )
54
55     # Calculate R2
56     r2 = r2_score(
57         eval_data["new_deaths_smoothed_2wk"],
58         eval_data[f"{name}_pred"]
59     )
60
61     # Save results
62     model_results.append({
63         "Model": name,

```

```

63         "Model": name,
64         "RMSE": rmse,
65         "R2": r2
66     })
67
68 # Create results table
69 results_table = pd.DataFrame(model_results)
70
71 # Display results
72 results_table

```

	Model	RMSE	R2	
0	Model 1	86.852994	-0.143774	
1	Model 2	54.047388	0.613674	
2	Model 3	85.942419	-0.142140	
3	Model 4	130.672825	-0.583339	
4	Model 5	90.612948	-0.137936	

Next steps:

[Generate code with results_table](#)

[New interactive sheet](#)

B. For your best model, calculate the Root Mean Squared Error for every country.

Hint: use `.groupby()`, then `.apply()` the RMSE to each group.

```

1 from sklearn.metrics import root_mean_squared_error
2 import numpy as np
3
4 def country_rmse(group):
5     group = group[["new_deaths_smoothed_2wk", "pred"]].dropna()
6
7     if len(group) == 0:
8         return np.nan
9
10    return root_mean_squared_error(
11        group["new_deaths_smoothed_2wk"],
12        group["pred"]
13    )
14
15 rmse_by_country = test.groupby("location").apply(
16     country_rmse,
17     include_groups=False
18 )
19

```

```

20 rmse_by_country = rmse_by_country.dropna().sort_values(ascending=False)
21
22 rmse_by_country

```

0	
location	
Japan	384.008487
France	170.797909
United States	53.355567
Australia	53.023239
Lithuania	40.573625
Estonia	37.879760
Sweden	32.995362
Ireland	32.051150
Malaysia	30.002362
Bulgaria	29.206677
Canada	26.065489
Belgium	25.596038
Netherlands	23.872063
Denmark	23.411008
Czechia	21.461256
Israel	19.357645
Italy	18.909979
Switzerland	18.080526
Austria	17.456199
Spain	5.237835

dtype: float64

✓ Conclusion

The regression modeling showed that current COVID case counts and hospitalization-related variables were the strongest predictions of future COVID deaths. Among the five models test, Model 2 performed the best with the lowest RMSE of 54.05 and the highest R^2 value of 0.614. This suggests that variable such as `new_cases_smoothed`, `hosp_patients`, and `total_vaccinations` were the most useful combination for predicting deaths two weeks into the future.

The transformed variables still provided useful insight into broader health and demographic conditions. Variables like `older_population` and `cardiovasc_deaths` may help explain differences between countries, even if they were not the strongest predictors in the regression model.

The country-level RMSE results also showed large differences between countries. Japan and France had the highest prediction errors, which may indicate more variability in COVID death trends or reporting patterns. Countries such as Spain and Austria had much lower RMSE values, suggesting the model predictions were more accurate for those countries.

Overall, the modeling suggests that short term COVID death rates are most strongly connected to recent infection levels and hospitalizations rather than long-term demographic characteristics alone.